

- Description

This module manages a smart window blind adjustment system, ensuring that:

1. The system continuously monitors the external brightness (in lux).
2. Based on the measured brightness, it adjusts the blind angle (in degrees) to optimize indoor light and temperature.
3. If the brightness is higher than a predefined high threshold, the blinds are lowered (angle increased) to reduce sunlight entry.
4. If the brightness is lower than a predefined low threshold, the blinds are raised (angle decreased) to allow more natural light.
5. If the brightness is within the optimal range, the blind angle is maintained at an intermediate setting.

This system helps optimize indoor lighting, reduce glare, and improve energy efficiency by controlling natural light.

- Specification

module SmartWindowBlindAdjustment

var currentBrightness, highBrightnessThreshold, lowBrightnessThreshold,
blindAngle: real

process updateBrightness(newBrightness: real) returns (blindAngle: real,
currentBrightness: real)

functional scenario form:

currentBrightness > highBrightnessThreshold && blindAngle = 80.0, Blinds lowered (80°) due to high brightness.

||

currentBrightness < lowBrightnessThreshold && blindAngle = 20.0, Blinds raised (20°) due to low brightness.

||

currentBrightness >= lowBrightnessThreshold && blindAngle = 50.0, Blinds set to 50°.

end process

end module

- Java Code

```
package code;
public class c77_SmartWindowBlind {
    private double highBrightnessThreshold;
    private double lowBrightnessThreshold;
    private double currentBrightness;
    private double blindAngle;
    public c77_SmartWindowBlind(double highThreshold, double lowThresho
ld, double initialBrightness, double initialAngle) {
        this.highBrightnessThreshold = highThreshold;
        this.lowBrightnessThreshold = lowThreshold;
```

```

        this.currentBrightness = initialBrightness;
        this.blindAngle = initialAngle;
    }
    public void updateBrightness(double newBrightness) {
        currentBrightness = newBrightness;
        adjustBlindAngle();
        System.out.printf("Current Brightness: %.2f lux, Blind Angle: %
.2f°\n", currentBrightness, blindAngle);
        if (currentBrightness > highBrightnessThreshold) {
            System.out.println("ALERT: High brightness detected. Blinds
lowered to reduce sunlight.");
        } else if (currentBrightness < lowBrightnessThreshold) {
            System.out.println("ALERT: Low brightness detected. Blinds
raised to allow more light.");
        } else {
            System.out.println("Brightness is optimal. Blinds maintaine
d at intermediate angle.");
        }
    }
    private void adjustBlindAngle() {
        if (currentBrightness > highBrightnessThreshold) {
            blindAngle = 80.0;
        } else if (currentBrightness < lowBrightnessThreshold) {
            blindAngle = 20.0;
        } else {
            blindAngle = 50.0;
        }
    }
}

public double getBlindAngle() {
    return blindAngle;
}

public double getCurrentBrightness() {
    return currentBrightness;
}

public static void main(String[] args) {
    c77_SmartWindowBlind blindSystem = new c77_SmartWindowBlind(800.0,
300.0, 500.0, 50.0);

    // testcase-VT:
    //     blindSystem.updateBrightness(900.0);
    //     blindSystem.updateBrightness(250.0);

```

```
//      blindSystem.updateBrightness(600.0);

//      // testcase-FT:
//      blindSystem.updateBrightness(3.832023139620673E294);
//      blindSystem.updateBrightness(-1.7976931348623157E308);
//      blindSystem.updateBrightness(1.6309011038425763E296);

//      // testcase-Z3:
//      blindSystem.updateBrightness(900.0);
//      blindSystem.updateBrightness(250.0);
//      blindSystem.updateBrightness(600.0);

//      // testcase-UVT:
//      blindSystem.updateBrightness(900.0);
//      blindSystem.updateBrightness(250.0);
//      blindSystem.updateBrightness(600.0);
    }
}
```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c77_SmartWindowBlind.java	100.0 %	8	0	8
✓ c77_SmartWindowBlind	100.0 %	8	0	8
main(String[])		0	0	0
c77_SmartWindowBlind(double,		0	0	0
adjustBlindAngle()	100.0 %	4	0	4
updateBrightness(double)	100.0 %	4	0	4

3. Vibration testing instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	104	20,950	21,054
✓ src	0.5 %	104	20,950	21,054
✓ code	0.5 %	104	20,950	21,054
✓ c77_SmartWindowBlind.java	94.5 %	104	6	110
✓ c77_SmartWindowBlind	94.5 %	104	6	110
main(String[])	100.0 %	18	0	18
c77_SmartWindowBlind(double,	100.0 %	15	0	15
adjustBlindAngle()	100.0 %	24	0	24
updateBrightness(double)	100.0 %	47	0	47

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	836	842
✓ src	0.7 %	6	836	842
✓ code	0.7 %	6	836	842
✓ c77_SmartWindowBlind.java	75.0 %	6	2	8
✓ c77_SmartWindowBlind	75.0 %	6	2	8
main(String[])		0	0	0
c77_SmartWindowBlind(double,		0	0	0
adjustBlindAngle()	75.0 %	3	1	4
updateBrightness(double)	75.0 %	3	1	4

5. Fuzz testing instruction coverage: 89.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	98	20,956	21,054
▼ src	0.5 %	98	20,956	21,054
▼ code	0.5 %	98	20,956	21,054
▼ c77_SmartWindowBlind.java	89.1 %	98	12	110
▼ c77_SmartWindowBlind	89.1 %	98	12	110
main(String[])	100.0 %	18	0	18
c77_SmartWindowBlind(double,	100.0 %	15	0	15
adjustBlindAngle()	87.5 %	21	3	24
updateBrightness(double)	93.6 %	44	3	47

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c77_SmartWindowBlind.java	100.0 %	8	0	8
▼ c77_SmartWindowBlind	100.0 %	8	0	8
main(String[])		0	0	0
c77_SmartWindowBlind(double,		0	0	0
adjustBlindAngle()	100.0 %	4	0	4
updateBrightness(double)	100.0 %	4	0	4

7. Z3 instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	104	20,950	21,054
▼ src	0.5 %	104	20,950	21,054
▼ code	0.5 %	104	20,950	21,054
▼ c77_SmartWindowBlind.java	94.5 %	104	6	110
▼ c77_SmartWindowBlind	94.5 %	104	6	110
main(String[])	100.0 %	18	0	18
c77_SmartWindowBlind(double,	100.0 %	15	0	15
adjustBlindAngle()	100.0 %	24	0	24
updateBrightness(double)	100.0 %	47	0	47

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c77_SmartWindowBlind.java	100.0 %	8	0	8
▼ c77_SmartWindowBlind	100.0 %	8	0	8
main(String[])		0	0	0
c77_SmartWindowBlind(double,		0	0	0
adjustBlindAngle()	100.0 %	4	0	4
updateBrightness(double)	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 94.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	104	20,950	21,054
▼ src	0.5 %	104	20,950	21,054
▼ code	0.5 %	104	20,950	21,054
▼ c77_SmartWindowBlind.java	94.5 %	104	6	110
▼ c77_SmartWindowBlind	94.5 %	104	6	110
main(String[])	100.0 %	18	0	18
c77_SmartWindowBlind(double,	100.0 %	15	0	15
adjustBlindAngle()	100.0 %	24	0	24
updateBrightness(double)	100.0 %	47	0	47